Spark vs. Ray: The Data Engineering Duel

Distributed ETL Benchmark on NYC Yellow Taxi Data

VedaVachan Reddy
DA3408 – Assignment 3
October 2026

This assignment compares Apache Spark and Ray on the same data preprocessing task. The aim is to see how both frameworks perform when working with a large dataset that contains normal cleaning operations as well as a Python-based transformation.

The pipeline used in both implementations was:

Read data → Clean → Join → Average Speed → Export

The experiment used **36 monthly NYC Yellow Taxi Parquet files**. The completed Ray run processed **128,202,548 rows**. Both frameworks were configured as distributed clusters with two workers.

The main result was that Ray successfully completed the complete pipeline in **348.59 seconds**. Spark started successfully and reached the Python transformation stage, but the Python workers crashed before the final export was completed.

Cluster Setup

Both Spark and Ray were configured with two workers as required by the assignment.

Spark Cluster

The Spark setup consisted of one Master and two Workers. Each worker had 2 CPU cores and 3 GB of configured memory, giving a total of 4 worker cores.

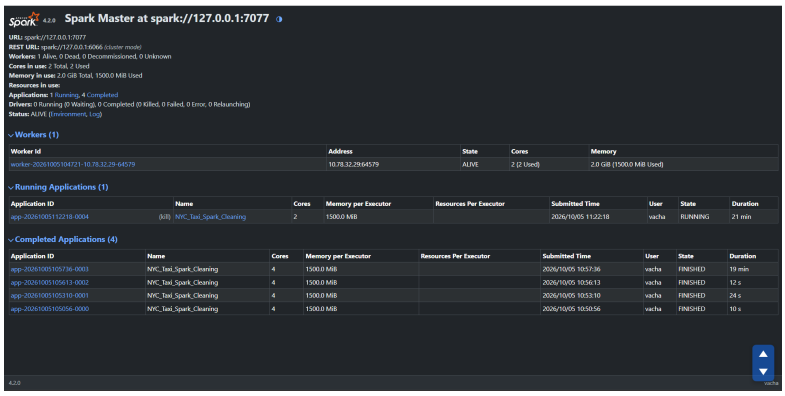


Figure 1: Spark Master showing two active workers.

The Spark application used Spark 4.2.0 with two executors.

Ray Cluster

The Ray setup used a Ray head node and two worker nodes. The cluster reported 4 CPUs and approximately 10.08 GiB of available memory.

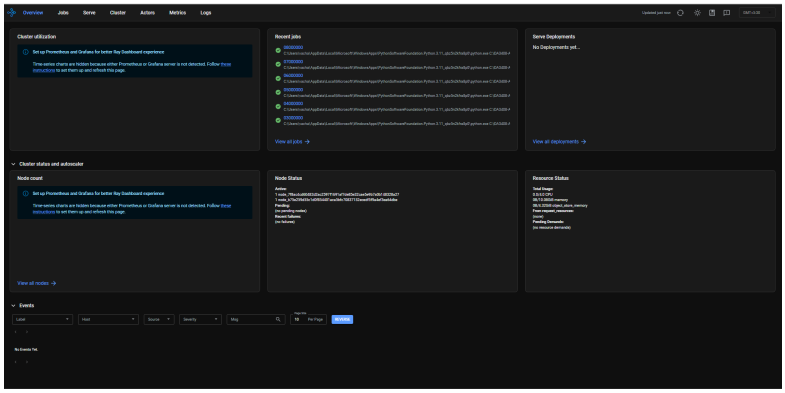


Figure 2: Ray Dashboard showing the distributed cluster.

Data Processing Pipeline

1. Ingestion: 36 monthly NYC Yellow Taxi Parquet files were read from the data directory.

2. Cleaning: Rows with missing required values were removed, duplicate records were removed, and pickup/drop-off timestamps were converted to timestamp format.

3. Taxi-zone join: The trip data was joined with the taxi-zone lookup table using pickup and drop-off location IDs. Spark used broadcast joins because the lookup table is small. Ray performed the lookup during its Python batch processing.

4. Average speed:

$$\boxed{\text{Average Speed} = \frac{\text{Trip Distance}}{\text{Trip Duration in Hours}}}$$

Invalid or non-positive trip durations were treated as invalid.

5. Export: The processed data was written to Parquet format.

Benchmark Method

The Ray benchmark completed successfully:

$$\boxed{\text{Ray time} = 348.59 \text{ seconds}}$$

This is approximately **5 minutes 49 seconds**.

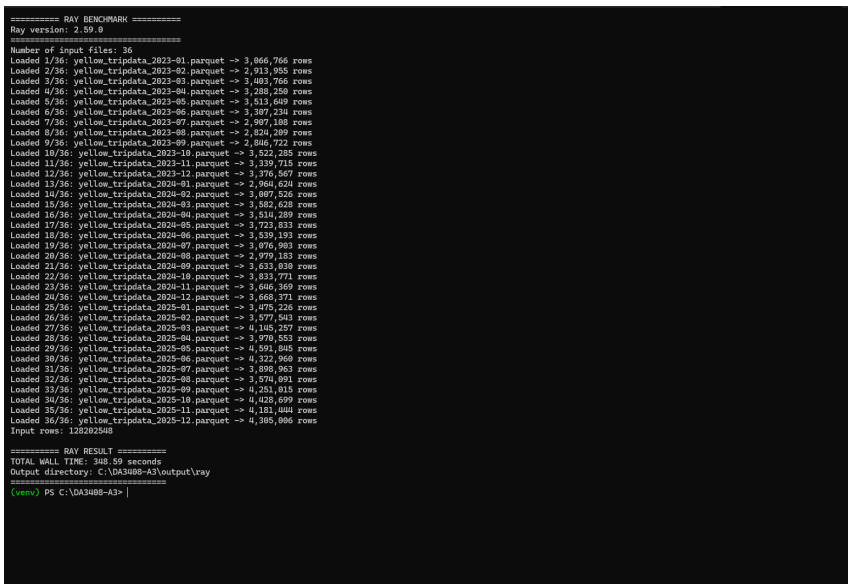


Figure 3: Ray benchmark output showing the completed run.

The Ray run loaded all 36 input files and processed 128,202,548 rows.

Spark also started successfully and registered both executors. However, during the Python UDF stage it reported:

```
Python worker exited unexpectedly (crashed)
```

The Spark log also showed a connection-aborted error between the Spark process and the Python worker. Because the final export did not complete, a valid Spark end-to-end execution time cannot be reported.

The Spark script printed these setup times:

- Ingestion setup: 12.96 s
- Cleaning setup: 0.55 s
- Join + UDF setup: 0.52 s

These are **not actual execution times**. Spark uses lazy evaluation, so the actual processing happens later when an action such as writing the output is performed.

Benchmark Results

Table 1 shows the actual results obtained from the experiments.

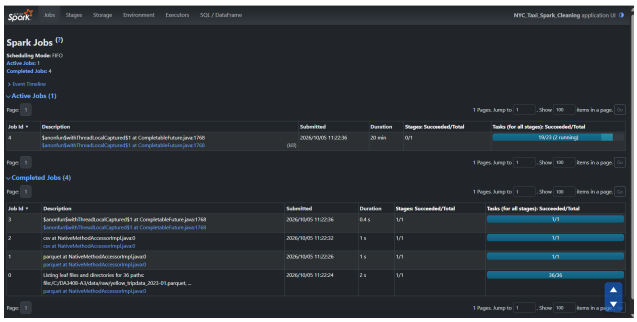
Table 1: Benchmark results

Metric	Spark	Ray
Input files	36	36
Input rows	128,202,548	128,202,548
End-to-end time	Not completed	348.59 s
Export	Failed	Completed
Python transformation	Failed	Completed
Peak CPU	Not captured	Not captured
Peak memory	Not captured	Not captured
Isolated UDF time	Not measured	Not measured

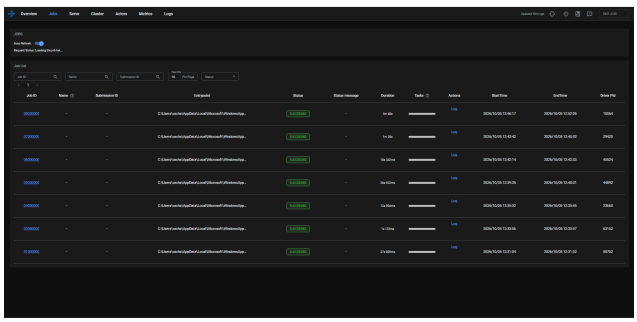
The most important observation is that Ray completed the complete pipeline, while Spark could not complete the final export.

0.1 Execution Evidence

The following screenshots provide additional evidence from the two frameworks.



(a) Spark application progress.



(b) Ray job history.

Figure 4: Execution evidence from Spark and Ray.

Python UDF Comparison

The average-speed calculation was the main Python-based transformation in the pipeline.

In Spark, it was implemented as a Python UDF inside the Spark dataframe pipeline. Spark uses a JVM-based execution engine, so Python UDFs require communication between the Spark execution system and the Python runtime.

In Ray, the same calculation was performed inside Python batch processing. This fits naturally with Ray's Python-based processing model.

In this experiment, the Spark Python workers crashed during this stage, while Ray completed the transformation successfully.

The isolated UDF time was not separately measured. Therefore, it is reported as **Not measured**.

The Spark value of 0.52 seconds should also not be considered the UDF runtime because it only represents the time used to construct the lazy transformation.

Discussion

0.2 Winner: Ray

For this particular experiment, **Ray is the winner**.

Ray completed the full pipeline in 348.59 seconds, including ingestion, cleaning, transformation, and Parquet export.

Spark successfully created the distributed cluster and started processing, but its Python workers crashed during the transformation stage. As a result, the final export was not completed.

This result is specific to this workload and the tested Windows environment. It does not mean that Ray will always be faster than Spark.

0.3 AI-first Workloads

For an AI-first project, I would choose **Ray** based on this experiment. The workload contains a Python-heavy transformation, and Ray's Python-based processing model fits well with Python and machine-learning workflows.

0.4 BI-first Workloads

For a BI-first project, **Spark** would still be a strong choice. Spark is well suited for structured data processing, dataframe operations and SQL-style analytics.

If a workload mainly uses normal dataframe and SQL operations with fewer custom Python functions, Spark would be a natural option.

Resource Measurements

Peak CPU and peak memory were not recorded while the actual benchmark was running. A PowerShell measurement taken after the benchmark cannot be used as the benchmark peak.

Therefore, no CPU or memory values have been invented.

The isolated Python UDF time was also not separately measured.

Another limitation is that Spark did not finish its complete pipeline. Because of this, a numerical Spark-versus-Ray speedup cannot be calculated.

These limitations are reported so that the benchmark results remain based only on measurements that were actually obtained.

Performance Tuning Note

AI assistance was used during development to help translate the processing logic between Spark and Ray and to troubleshoot distributed execution and Python-worker issues.

The cluster and network configuration were set up manually.

For Spark, broadcast joins were used because the taxi-zone lookup table is small. Adaptive execution and controlled shuffle partitions were also used.

For Ray, the lookup operation was performed during the distributed Python batch processing.

No synthetic benchmark numbers were used in this report.

Conclusion

This experiment compared Spark and Ray using the same NYC Yellow Taxi preprocessing task.

Ray successfully completed the complete pipeline in:

$$\boxed{348.59 \text{ seconds}}$$

or approximately 5 minutes and 49 seconds.

Spark successfully established its two-worker cluster and started the processing, but its Python worker crashed during execution and the final export did not complete.

Therefore, **Ray is the winner for this particular benchmark**. Ray is especially suitable for the Python-heavy and AI-oriented workload used in this experiment, while Spark remains a strong option for traditional BI and structured-data processing.

Overall, the experiment shows that the best framework depends on the type of workload and execution environment.